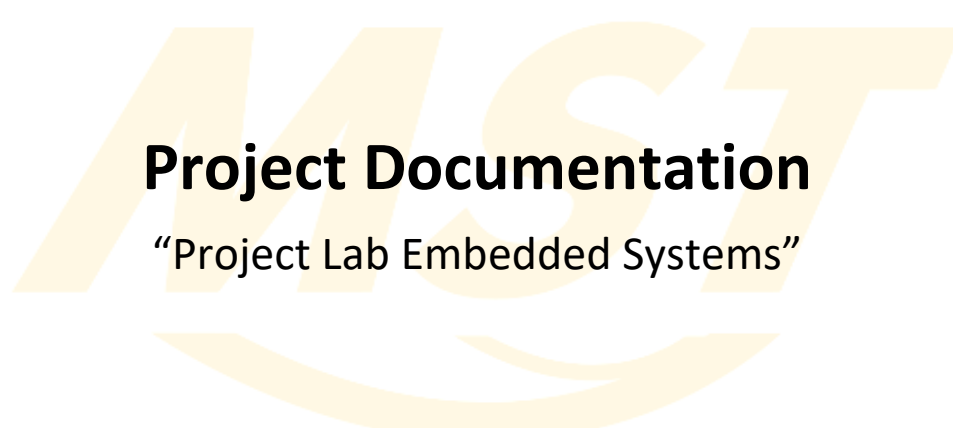


TECHNISCHE UNIVERSITÄT
CHEMNITZ

Department of Electrical Engineering and Information
Technology

Chair of Measurement and Sensor Technology



Project Documentation

“Project Lab Embedded Systems”

Group No.:

33

Members:

Karan Ganesh Mahadik - 893310

Shreya Rajendra Desai - 908966

Jyothilakshmi Mohan - 909301

Project: Adaptation of MATLAB GUI for Wireless EIT Data Acquisition via BLE

Date: 2026-08-06

Table of Contents

1. Abstract	03
2. Member Responsibilities	03
3. Functional Description	03
3.1 Overview	03
3.2 Software	04
3.3 Hardware	09
3.4 Interaction between User, Hardware and Software.....	12
3.5 Demonstration Screenshots	13
3.6 Testing and Troubleshooting	15
4. Description of Files	16
5. References	17

1 Abstract

The focus of this project is to have a gesture data collection system based on Electrical Impedance Tomography (EIT). In EIT, electrical currents are injected into a body (or object), and resulting voltages are measured to identify the conductivity pattern, based on the finger and hand position the voltage pattern changes. The interfacing between Arduino system and MATLAB GUI is done by using Bluetooth HC05 wireless communication which is a low-cost serial communication device, and it provides the wireless serial link between the 2 systems. The Arduino system performs the desired EIT scan and the measured voltage value is transmitted to MATLAB GUI. The MATLAB Graphical User Interface (GUI) provides the user with all information regarding the selection of gesture to perform data collection, to confirm the scan type, to start or stop the data collection. Also, in MATLAB GUI the measured voltage corresponding to the gesture is displayed. Moreover, MATLAB removes noise from the data, applies filtering, makes data structured and is finally stored in horizontal CSV format.

2 Member Responsibilities

Karan Ganesh Mahadik MATLAB UI Design, CSV data acquisition, Python port scan
Shreya Rajendra Desai Arduino firmware and Bluetooth communication.
Jyothilakshmi Mohan MATLAB UI design, Hardware testing, documentation.

3 Functional Description

3.1 Overview

The objective of this project was to modify an existing MATLAB Graphical User Interface (GUI) so that it communicates with the Electrical Impedance Tomography (EIT) system using Bluetooth Low Energy (BLE) instead of a wired serial connection. The wireless interface enables the user to discover nearby BLE devices, establish a connection with the EIT hardware, configure measurement parameters, receive impedance measurement data, and display the results in real time.

The system consists of four major components:

- MATLAB GUI running on the computer
- Arduino Uno
- Bluetooth Module (HC-05)
- EIT hardware
- User interface for visualization and control

The user first launches the MATLAB application, scans for available BLE devices, connects to the EIT system, selects the desired measurement configuration, and starts data acquisition. The received impedance measurements are processed and displayed continuously on the GUI.

3.2 Software

3.2.1 The software was developed in MATLAB using the Bluetooth Low Energy Toolbox. The existing serial communication layer was replaced with BLE communication while preserving the original graphical interface and visualization functions.

The software performs the following tasks:

Software Part	Function
Arduino firmware	Receives commands, selects scan type, performs measurement, and sends frames.
MATLAB UI	Controls data collection, gesture display, plotting, and CSV writing.
Python scanner	Searches the HC-05 COM port and writes last_hc05_port.txt.
CSV files	Store Type 1, Type 2, Type 3, and Type 4 datasets separately.

Table 1: Software description

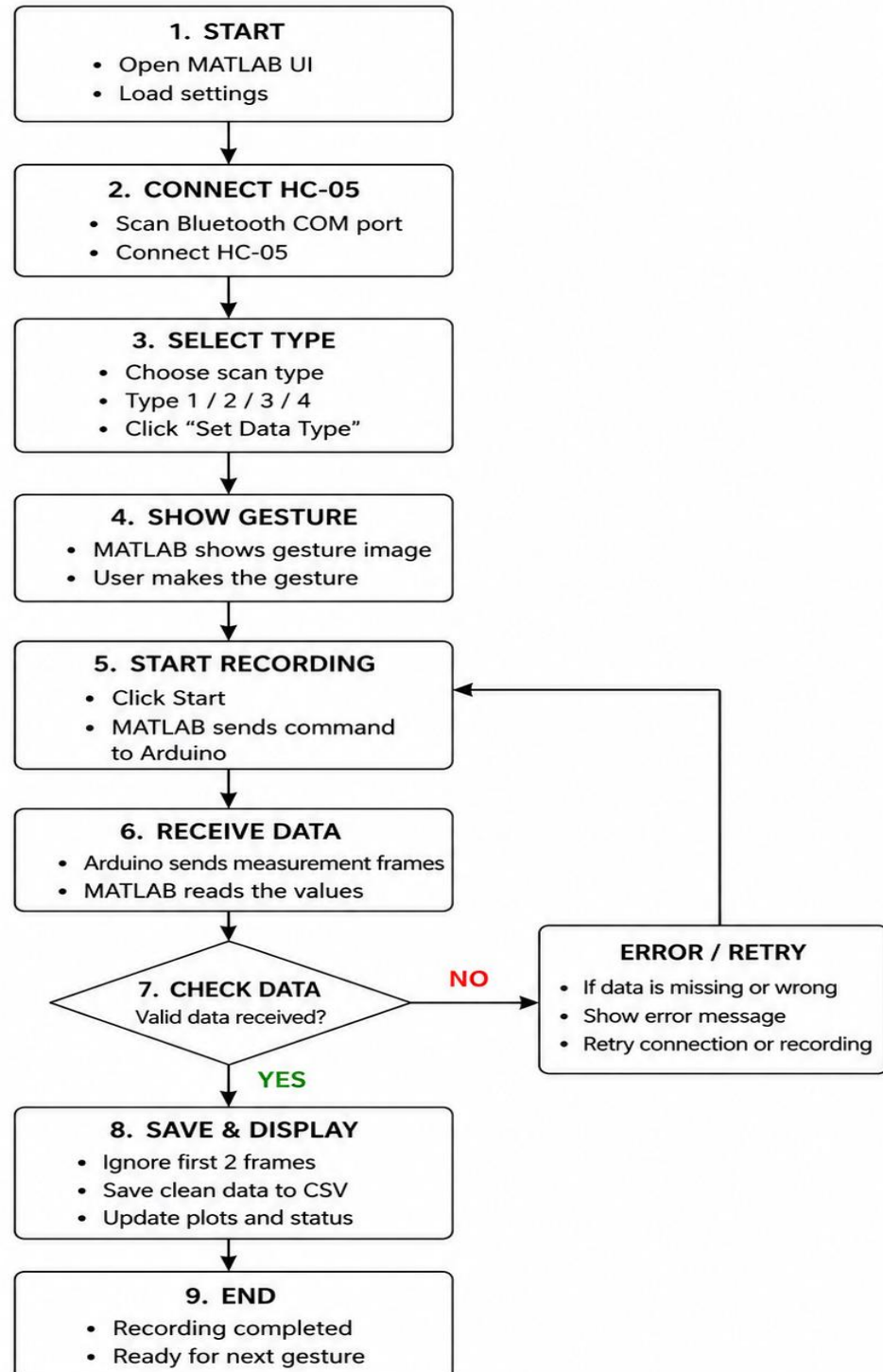


Fig. 1: Software Flowchart (MATLAB-BLE Communication)

3.2.2 MATLAB Initialization and Connection

The main MATLAB function loads project settings, creates the graphical interface and prepares all recording state variables. Recording is enabled only after the HC-05 connection and scan-type confirmation are successful.

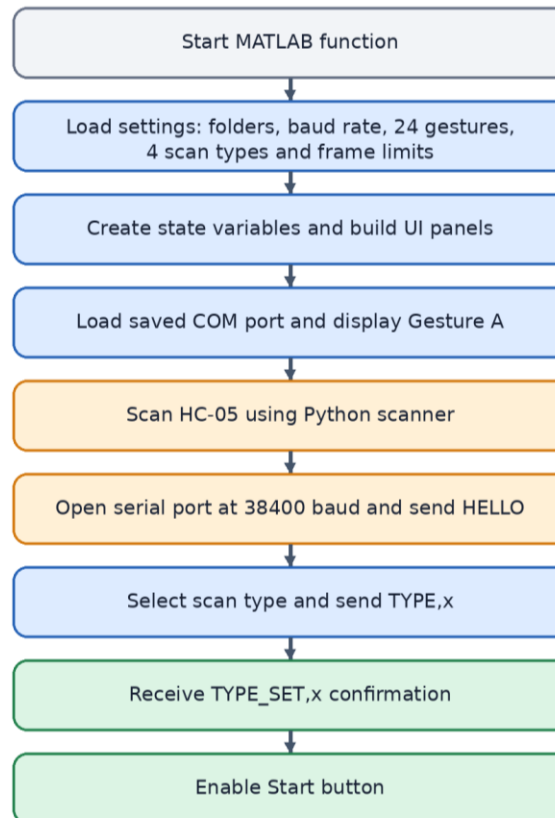


Fig. 2: MATLAB Initialization and Connection Flow

Function	Purpose
scanForHC05Port()	Runs the Python scanner and reads the detected Bluetooth COM port.
connectHC05()	Opens the serial port, sends HELLO and installs the serial callback.
setDataIconButtonPushed()	Sends TYPE,1..4 and waits for TYPE_SET confirmation.
showCurrentGestureImage()	Loads and displays the current gesture image.

Table 2: Function description

3.2.3 MATLAB Recording, Processing and CSV Storage

When the **Start button** is pressed, MATLAB first checks whether the Arduino is connected and whether the selected EIT scan type has been confirmed using the TYPE_SET message. Recording begins only after Arduino confirms the correct scan pattern. For example, when Type 2 is selected, Arduino must send TYPE_SET,2.

After confirmation, MATLAB starts a new recording session by resetting the frame counters, clearing temporary variables, loading the CSV file for the selected scan type, and storing the gesture currently displayed on the screen. This gesture image becomes the label for the recorded data. Arduino does not need to know the gesture name; it only performs the selected EIT scan and sends voltage values to MATLAB.

MATLAB then sends the TYPE,x command to select Scan Type 1, 2, 3, or 4, followed by the N command to start the measurement. Arduino sends each complete voltage frame as one comma-separated line through the Bluetooth serial connection. The number of values per frame is:

- Type 1: 40 values
- Type 2: 16 values
- Type 3: 32 values
- Type 4: 120 values

During recording, MATLAB receives each frame using the serial callback function and checks whether it contains the expected number of values. Arduino sends 12 frames for each gesture, but MATLAB ignores the first two frames because they may be unstable due to multiplexer switching and signal settling. Therefore, only Frames 3 to 12 are used as valid measurement data.

For each valid frame, MATLAB assigns the current gesture label, updates the live voltage and channel plots, and saves the data as one horizontal row in the selected CSV file. The format is:

R1, R2, ..., Rn, GestureName

Separate CSV files are maintained for each scan type so that the different measurement patterns are not mixed. Since ten valid frames are saved for each gesture, MATLAB adds ten rows to the selected CSV file, finishes the current recording, automatically loads the next gesture, and prepares the system for the next measurement.

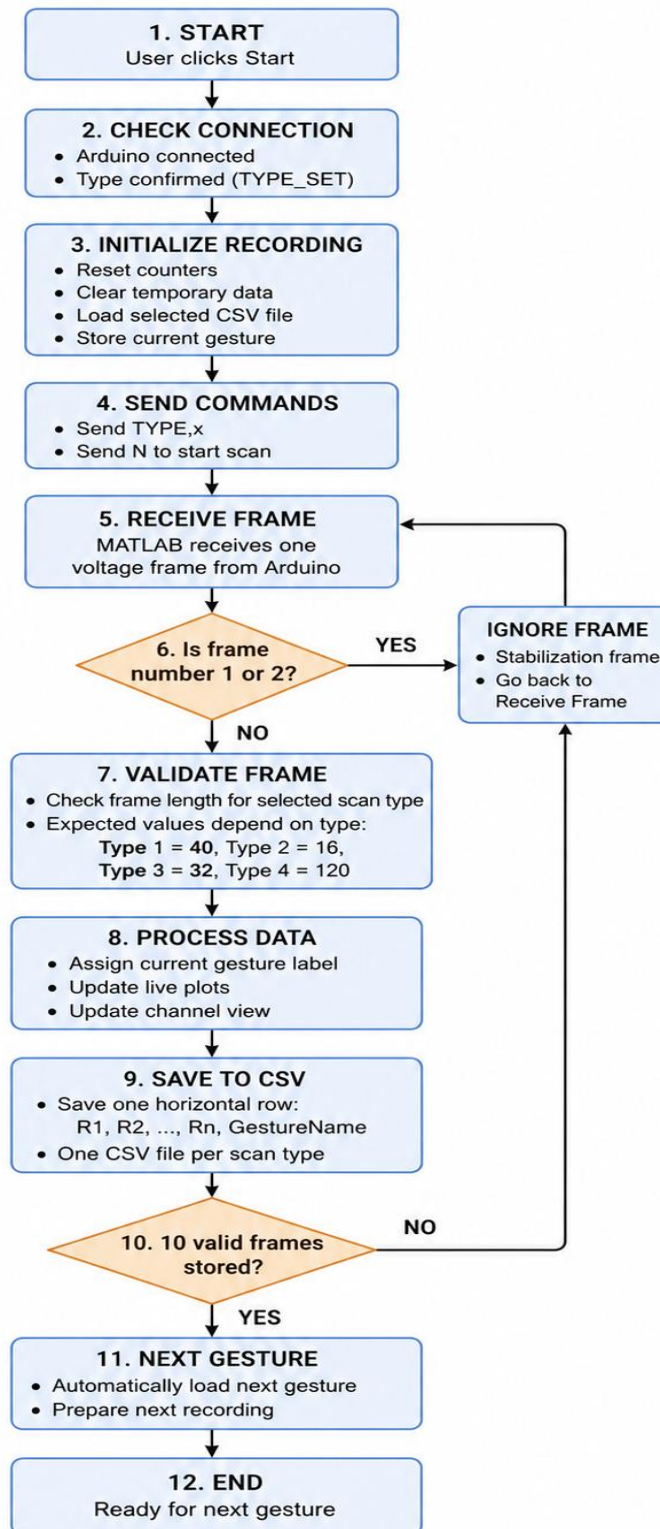


Fig. 3: MATLAB Recording, Processing and CSV Storage block diagram

3.2.4 Data Acquisition and CSV Format

Arduino sends one complete frame in a single serial line. This line contains multiple comma-separated values, not just one value. For example, Type 2 sends one line with 16 readings. MATLAB splits the line at each comma, converts the values to numbers, and stores them as one horizontal CSV row. Therefore, one Arduino frame line becomes one saved CSV row.

Scan Type	Pattern Name	Values per Frame	CSV File Example
Type 1	Adjacent	40	dataset_Type1_Adjacent.csv
Type 2	Opposite	16	dataset_Type2_Opposite.csv
Type 3	Skip-1	32	dataset_Type3_Skip1.csv
Type 4	Rotating Radial	120	dataset_Type4_RR.csv

Table 3: Data format

For each recording, Arduino sends 12 frame lines. MATLAB ignores frame 1 and frame 2 because these frames may contain unstable values immediately after switching. Frames 3 to 12 are saved. Hence, one gesture recording produces 10 clean horizontal rows for the selected scan type.

Frame Number	MATLAB Action
Frame 1	Ignored as stabilization/garbage frame.
Frame 2	Ignored as stabilization/garbage frame.
Frame 3 to Frame 12	Saved as 10 clean horizontal CSV rows.

Table 4: MATLAB Action

3.3 Hardware

The hardware used in this project consists of the Electrical Impedance Tomography (EIT) measurement system equipped with Bluetooth communication capability.

A. Hardware Components

- Electrical Impedance Tomography (EIT) System

- Bluetooth module (HC-05)
- Personal computer running MATLAB
- Power supply
- Measurement electrodes

B. Hardware Operation

The EIT hardware generates electrical impedance measurements from the connected electrodes. These measurements are transmitted wirelessly through the BLE module. MATLAB receives the data, processes it, and displays the results.

C. Requirements

The hardware should satisfy the following requirements:

- Stable BLE communication
- Reliable data transmission
- Continuous measurement capability
- Real-time response
- Low communication delay

D. Verification

The hardware was tested by

- Discovering the BLE device
- Establishing a wireless connection
- Reading BLE characteristics
- Receiving measurement packets
- Comparing received data with the original wired implementation

The successful tests confirmed that the BLE interface functions correctly for real-time EIT data acquisition.

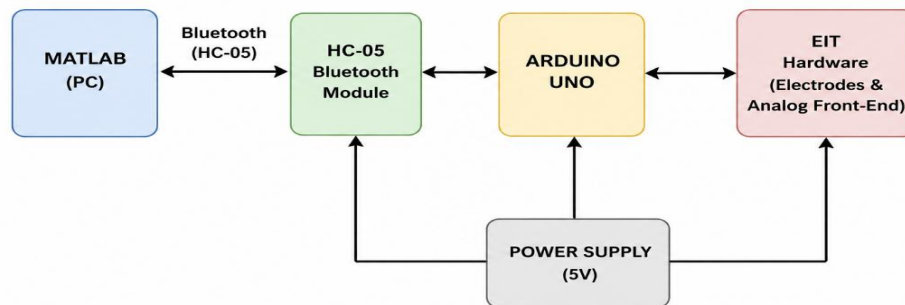


Fig. 4: System Hardware

3.4 Interaction between User, Hardware and Software

The interaction between the user, MATLAB software, BLE communication layer, and EIT hardware follow the sequence below.

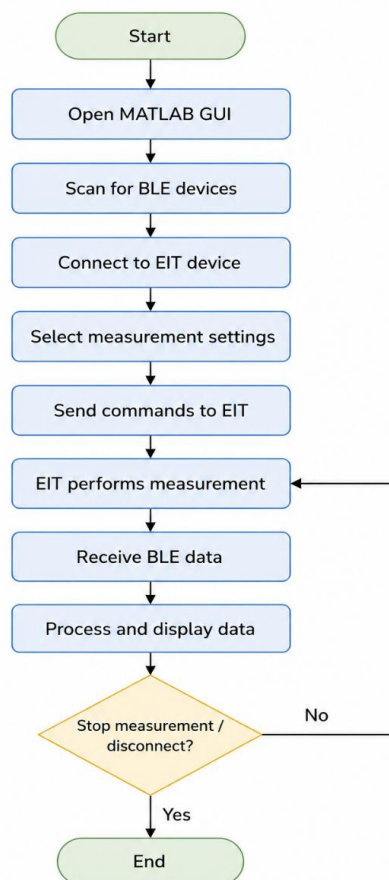


Fig. 5: Interaction between User, Hardware and Software

3.5 Demonstration Screenshots

The following screenshots were extracted from the recorded system demonstration and illustrate the practical operation of the developed MATLAB user interface. They present the complete workflow, including Bluetooth connection establishment, scan-type selection, gesture visualization, real-time measurement plotting, data acquisition, and verification of the recorded CSV files. These screenshots provide evidence that the software functions were successfully integrated and validated under actual operating conditions.

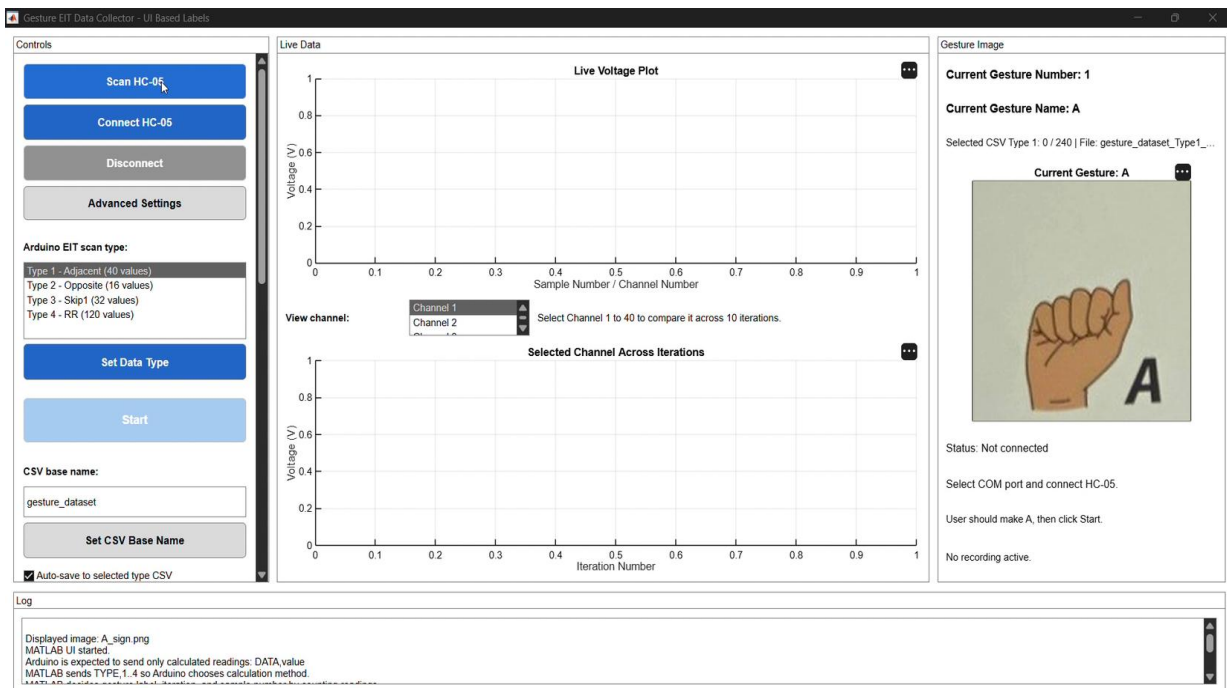


Fig. 6: MATLAB UI startup with HC-05 scan and Gesture A display.

The startup interface provides the initial control environment for the measurement workflow. The control panel enables HC-05 Bluetooth device scanning and connection establishment, while the gesture display panel provides a visual reference of the selected gesture to be performed by the user. This ensures correct user interaction and prepares the system for synchronized data acquisition.

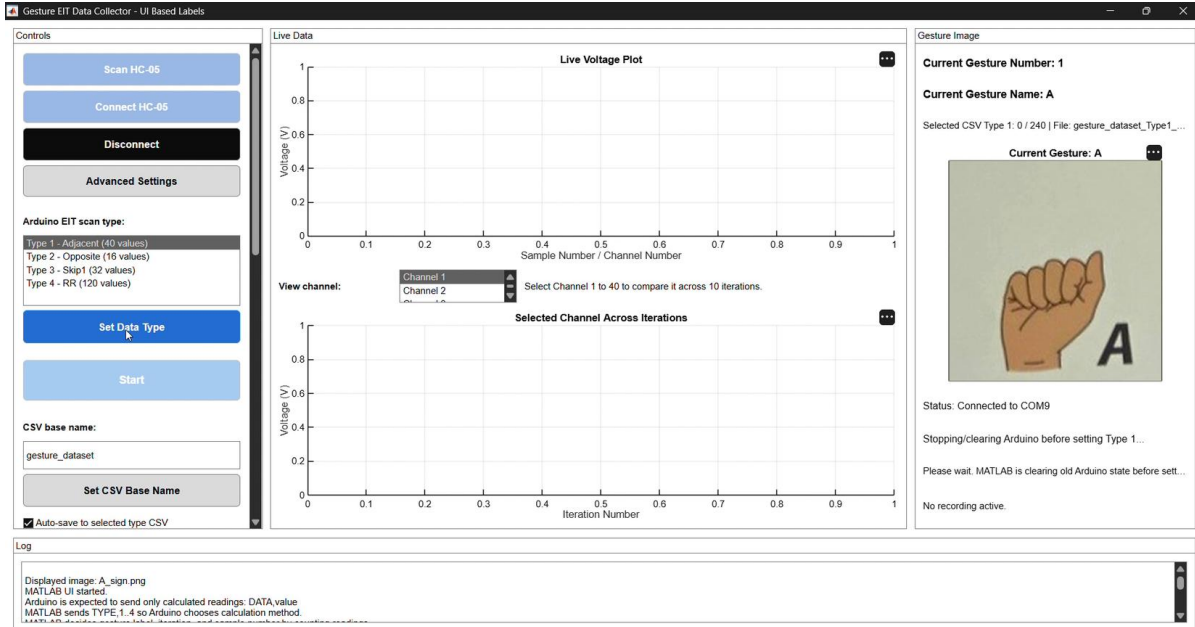


Fig. 7: HC-05 connection established and MATLAB waiting for scan type selection.

After the HC-05 connection is established, MATLAB waits for scan type confirmation. This prevents recording with a wrong scan pattern.

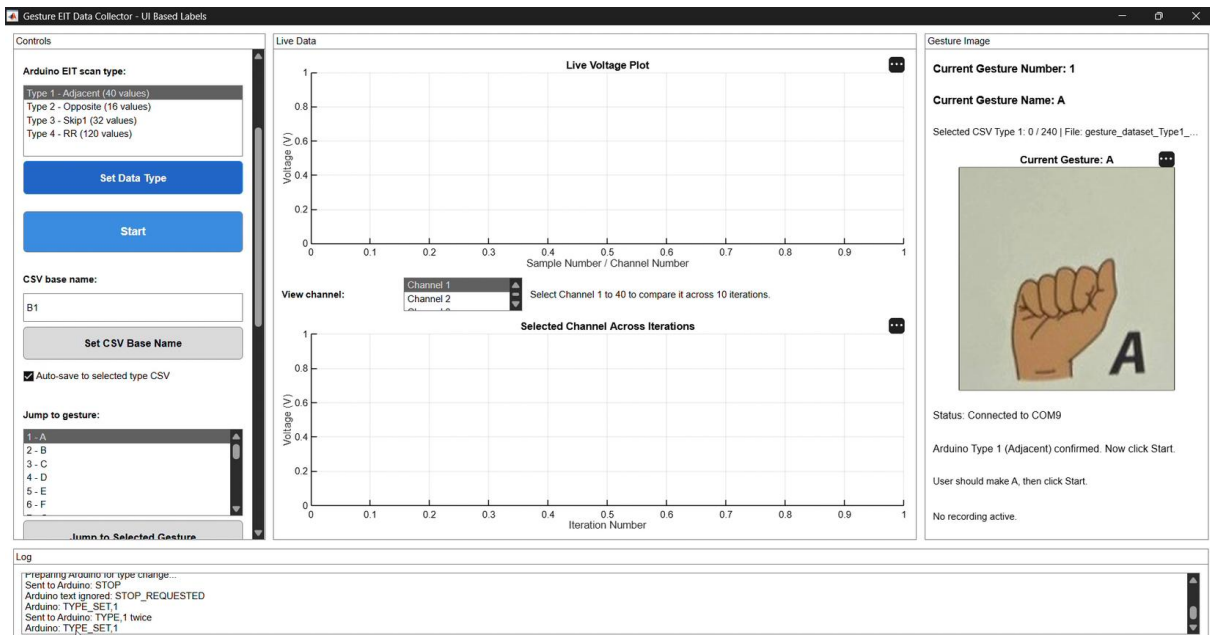


Fig. 8: MATLAB controls for selecting and confirming the EIT scan type.

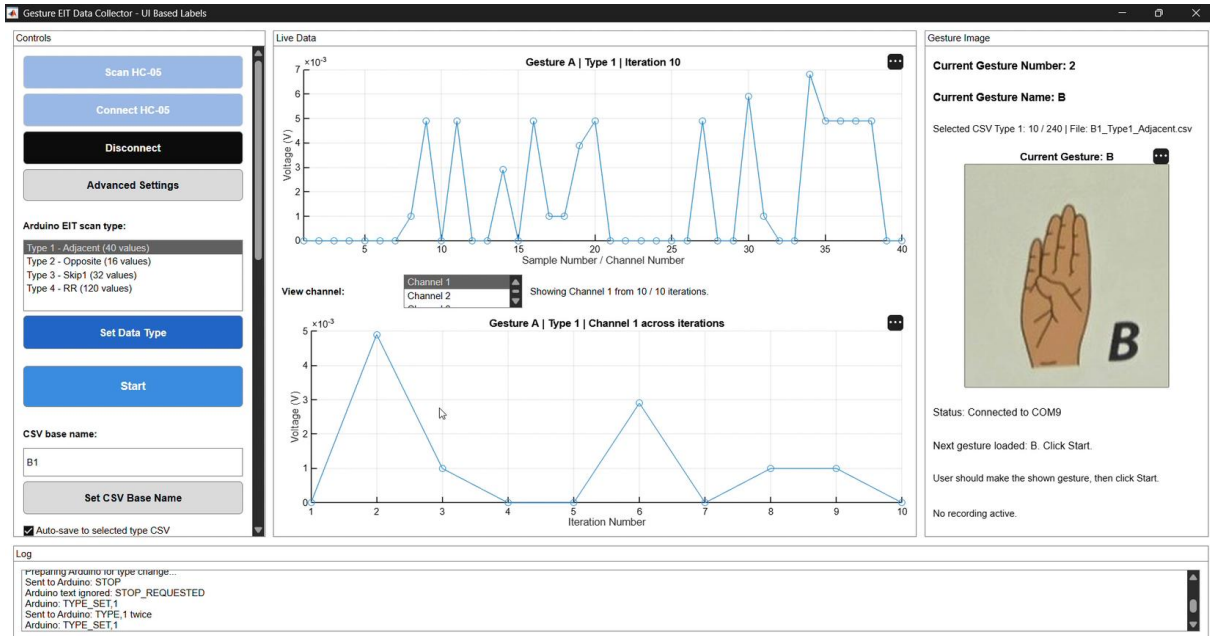


Fig. 9: Live data acquisition and plotting during gesture recording.

During recording, the live voltage plot updates with the current frame. The lower plot shows the selected channel across the saved iterations.

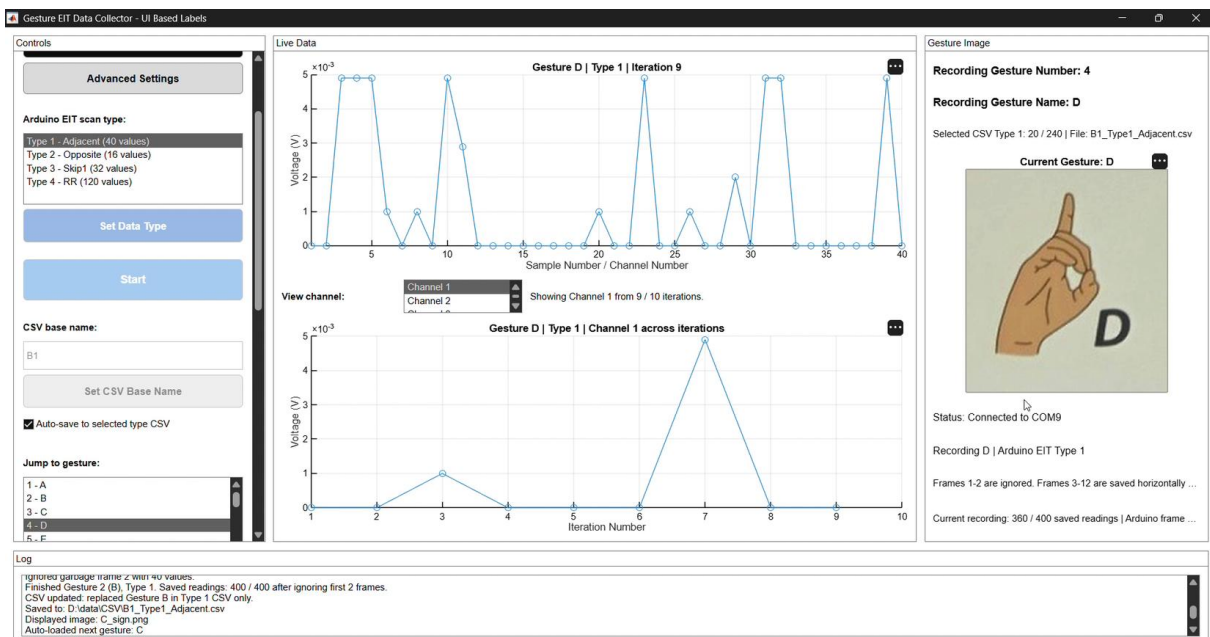


Fig. 10: Recording workflow with gesture image, voltage plot, and status messages.

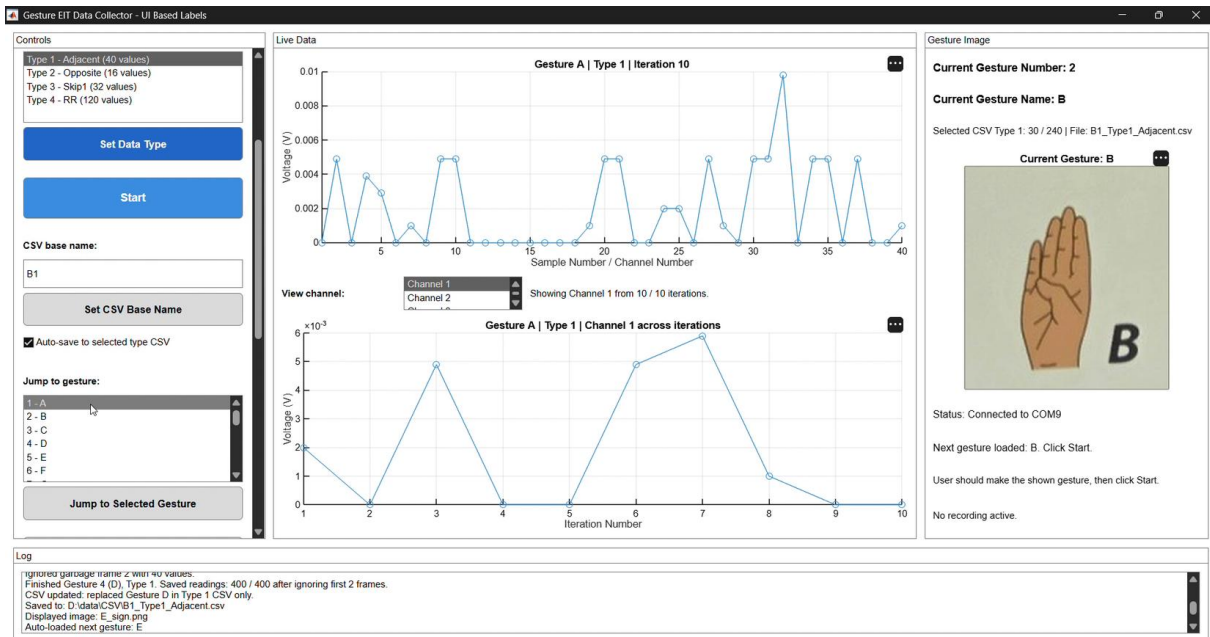


Fig. 13: Continued recording demonstration with selected-channel visualization.

3.6 Testing and Troubleshooting

Testing focused on communication reliability, correct scan type selection, stable frame reception, and correct CSV output. The most important problem observed during testing was attempting to change the scan type while Arduino was still recording. In that case, Arduino rejected the command because changing type during active recording can corrupt the dataset. The final safe workflow requires MATLAB to wait for recording completion and TYPE_SET confirmation before enabling Start.

Observed Issue	Reason	Solution
HC-05 not found	Bluetooth not paired or wrong COM port.	Pair HC-05 in Windows and run the Python scanner.
Waiting for type confirmation	Arduino has not yet replied TYPE_SET,x.	Keep Start disabled and wait for confirmation.
Expected 16 values but received 40	MATLAB selected Type 2 but Arduino was still in Type 1.	Set Data Type again and wait for TYPE_SET,2 before recording.
Cannot change type while recording	Arduino is still completing the previous	Wait until END or use STOP/RESET before selecting a new type.

Observed Issue	Reason	Solution
	measurement sequence.	
Readings appear without electrodes	Analog input is floating and picking up noise.	Connect the measurement circuit and electrodes before collecting final data.

Table 5: Testing and Troubleshooting

4. Description of Files

The project files are organized into Arduino source code, MATLAB UI source code, the Python scanner, gesture images, and generated CSV files. A clear folder structure is required to ensure that MATLAB can find the gesture images, write the dataset files, and read the detected HC-05 COM port.

File / Folder	Content
Arduino .ino file	Stable Arduino firmware for HC-05 communication and EIT frame transmission.
MATLAB .m file	Bluetooth MATLAB UI for scanning, connecting, plotting, and CSV storage.
scan_hc05.py	Python scanner used to detect the HC-05 COM port.
last_hc05_port.txt	Text file storing the last detected Bluetooth COM port.
D:\data\gestures	Folder containing gesture images such as A_sign.png, B_sign.png, and others.
D:\data\CSV	Folder where Type 1, Type 2, Type 3, and Type 4 CSV files are saved.

Table 6: File description

Conclusion

The Bluetooth HC-05 interface was successfully integrated with the Arduino and MATLAB-based EIT gesture data collection system. The developed interface provides a controlled recording process by checking the Arduino connection and confirming the selected scan type before measurement begins. It also displays the gesture image, receives live voltage frames, and updates the voltage and channel plots during recording.

To improve data quality, the first two unstable frames are ignored, while the remaining valid frames are labelled and stored in a horizontal CSV format. Separate CSV files are maintained

for each scan type, which keeps the dataset organized and prevents different EIT measurement patterns from being mixed.

Overall, the system offers reliable wireless communication, automatic gesture labelling, real-time visualization, and structured data storage. It provides a suitable foundation for future EIT gesture recognition, data analysis, and machine learning applications.

6 References

<https://www.feasycom.com/datasheet/fsc-hc05.pdf>

<https://docs.arduino.cc/hardware/uno-r3>

<https://www.mathworks.com/help/matlab/>